

DS ASSIGNMENT SET 2

-CRISTINA JANE.B

1. Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

```
#include <stdio.h>

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr;

    ptr = arr;

    printf("Original array elements:\n");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }

    for (int i = 0; i < 5; i++)
    {
        *(ptr + i) = *(ptr + i) + 5;
    }

    printf("\n\nModified array elements:\n");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }

    return 0;
}
```

OUTPUT:

```

PS C:\Users\B.Cristina Jane\OneDrive\Desktop\ds assignment 2> ./a
Original array elements:
10 20 30 40 50

Modified array elements:
15 25 35 45 55

```

2. Calculate string length using pointer.

```
#include <stdio.h>
```

```

int main()
{
    char str[100];
    char *ptr;
    int length = 0;

    printf("Enter a string: ");
    scanf("%[^\n]", str);

    ptr = str;

    while (*ptr != '\0')
    {
        length++;
        ptr++;
    }

    printf("Length of the string = %d", length);

    return 0;
}

```

OUTPUT:

```

PS C:\Users\B.Cristina Jane\OneDrive\Desktop\ds assignment 2> ./a
Enter a string: hello
Length of the string = 5

```

3. Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

```
#include <stdio.h>
```

```

void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}

int main()
{
    int a, b;
    int sum, product;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

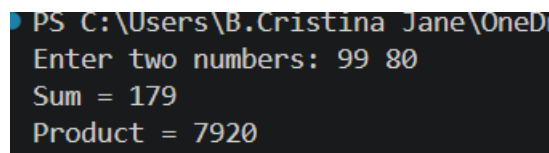
    calculate(a, b, &sum, &product);

    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);

    return 0;
}

```

OUTPUT:



```

PS C:\Users\B.Cristina Jane\OneD
Enter two numbers: 99 80
Sum = 179
Product = 7920

```

4.Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Student
```

```
{
```

```
    int roll;
```

```
    char name[20];
```

```
float mark;

};

int main()
{
    int n, i;
    struct Student *ptr;

    printf("Enter number of students: ");
    scanf("%d", &n);

    ptr = (struct Student *)malloc(n * sizeof(struct Student));

    for (i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d:\n", i + 1);

        printf("Roll: ");
        scanf("%d", &(ptr + i)->roll);

        printf("Name: ");
        scanf("%s", (ptr + i)->name);

        printf("Mark: ");
        scanf("%f", &(ptr + i)->mark);
    }

    for (i = 0; i < n; i++)
```

```
{  
    (ptr + i)->mark = (ptr + i)->mark + 5;  
}  
  
printf("\nStudent Details:\n");  
  
for (i = 0; i < n; i++)  
{  
    printf("\nRoll : %d", (ptr + i)->roll);  
    printf("\nName : %s", (ptr + i)->name);  
    printf("\nMark : %.2f\n", (ptr + i)->mark);  
}  
  
free(ptr);  
  
return 0;  
}
```

OUTPUT:

```

Enter number of students: 3

Enter details of student 1:
Roll: 52
Name: jane
Mark: 100

Enter details of student 2:
Roll: 50
Name: moni
Mark: 99

Enter details of student 3:
Roll: 13
Name: hari
Mark: 77

Student Details:

Roll : 52
Name : jane
Mark : 105.00

Roll : 50
Name : moni
Mark : 104.00

Roll : 13
Name : hari
Mark : 82.00
PS C:\Users\B.Cristina Jane\OneDrive\De

```

5. Define two functions with the same signature. Declare a function pointer and use the pointer to call both functions.

```
#include <stdio.h>
```

```
void display1()
```

```
{
    printf("This is Function 1\n");
}
```

```
void display2()
```

```
{
    printf("This is Function 2\n");
}
```

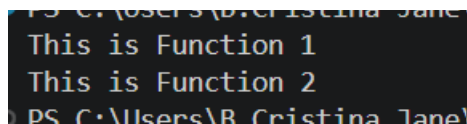
```
int main()
{
    void (*ptr)();

    ptr = display1;
    ptr();

    ptr = display2;
    ptr();

    return 0;
}
```

OUTPUT:



```
PS C:\Users\B.Cristina-Jane>
This is Function 1
This is Function 2
PS C:\Users\B.Cristina-Jane>
```

6. Declare an integer pointer to it and a double pointer. Print the values and addresses at each level.

```
#include <stdio.h>
```

```
int main()
{
    int x = 10;

    int *ptr;
```

```
int **dptr;

ptr = &x;
dptr = &ptr;

printf("Value of x = %d\n", x);
printf("Address of x = %p\n", (void *)&x);

printf("\nUsing integer pointer:\n");
printf("Value of ptr = %p\n", (void *)ptr);
printf("Value pointed by ptr = %d\n", *ptr);
printf("Address of ptr = %p\n", (void *)&ptr);

printf("\nUsing double pointer:\n");
printf("Value of dptr = %p\n", (void *)dptr);
printf("Value pointed by dptr = %p\n", (void *)*dptr);
printf("Value pointed by **dptr = %d\n", **dptr);
printf("Address of dptr = %p\n", (void *)&dptr);

return 0;
}
```

OUTPUT:


```
Value of x = 10
Address of x = 0000005C311FF75C

Using integer pointer:
Value of ptr = 0000005C311FF75C
Value pointed by ptr = 10
Address of ptr = 0000005C311FF750

Using double pointer:
Value of dptr = 0000005C311FF750
Value pointed by dptr = 0000005C311FF75C
Value pointed by **dptr = 10
Address of dptr = 0000005C311FF748
```

7. Sort an array of floating-point numbers using Selection Sort.

```
#include <stdio.h>
```

```
int main()
{
    int n, i, j, min;
    float arr[100], temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the floating-point numbers:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%f", &arr[i]);
    }

    for (i = 0; i < n - 1; i++)
    {
        min = i;
```

```
    for (j = i + 1; j < n; j++)  
    {  
        if (arr[j] < arr[min])  
        {  
            min = j;  
        }  
    }  
}
```

```
temp = arr[i];  
arr[i] = arr[min];  
arr[min] = temp;  
}
```

```
printf("Sorted array:\n");  
for (i = 0; i < n; i++)  
{  
    printf("%.2f ", arr[i]);  
}
```

```
return 0;  
}
```

OUTPUT:

```
Enter number of elements: 5  
Enter the floating-point numbers:  
31.2 53.4 99.6 71.4 73.6  
Sorted array:  
31.20 53.40 71.40 73.60 99.60
```

8. Define a structure Student with fields id, name and grade. Create an array of student structures and sort them based on different fields.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Student
```

```
{
```

```
    int id;
```

```
    char name[30];
```

```
    float grade;
```

```
};
```

```
int main()
```

```
{
```

```
    struct Student s[5], temp;
```

```
    int i, j, choice;
```

```
    printf("Enter details of 5 students:\n");
```

```
    for (i = 0; i < 5; i++)
```

```
    {
```

```
        printf("\nStudent %d\n", i + 1);
```

```
        printf("ID: ");
```

```
        scanf("%d", &s[i].id);
```

```
        printf("Name: ");
```

```

scanf("%s", s[i].name);

printf("Grade: ");
scanf("%f", &s[i].grade);
}

printf("\nSort based on:\n");
printf("1. ID\n");
printf("2. Name\n");
printf("3. Grade\n");

printf("Enter your choice: ");
scanf("%d", &choice);
for (i = 0; i < 4; i++)
{
    for (j = i + 1; j < 5; j++)
    {
        if ((choice == 1 && s[i].id > s[j].id) ||
            (choice == 2 && strcmp(s[i].name, s[j].name) > 0) ||
            (choice == 3 && s[i].grade > s[j].grade))
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
    }
}

```

```
printf("\nSorted Student Details:\n");
```

```
for (i = 0; i < 5; i++)
```

```
{
```

```
    printf("ID: %d Name: %s Grade: %.2f",
```

```
        s[i].id, s[i].name, s[i].grade);
```

```
    printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:

Enter details of 5 students:

Student 1

ID:

101

Name:

jane

Grade:

99.9

Student 2

ID:

102

Name:

gaja

Grade:

70.00

Student 3

ID:

103

Name:

sawan

Grade:

40.00

Student 4

ID:

104

Name:

alvin

Grade:

100.00

Student 5

```

ID:
  105
Name:
  kala
Grade:
  60.00

Sort based on:
1. ID
2. Name
3. Grade
Enter your choice: 1

Sorted Student Details:
ID: 101  Name: jane  Grade: 99.90
ID: 102  Name: gaja  Grade: 70.00
ID: 103  Name: sawan  Grade: 40.00
ID: 104  Name: alvin  Grade: 100.00
ID: 105  Name: kala  Grade: 60.00
PS C:\Users\B.Cristina Jane\OneDrive\Desk

```

9. Sort a set of negative numbers using Radix Sort.

```
# include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[6]={15,45,-1,2,231,480};
```

```
    int largest=0,nop=0,remainder=0,divisor=1;
```

```
    int bucketcount[10],bucket[10][10],min=0;
```

```
    printf("before sorting\n");
```

```
    for(int i=0;i<6;i++)
```

```
    {
```

```
        printf("%d\n",arr[i]);
```

```
    }
```

```
    for(int i=0;i<6;i++)
```

```
    {
```

```
        if(min>arr[i])
```

```
{
    min=arr[i];
}
}
for(int i=0;i<6;i++)
{
    arr[i]=arr[i]-min;
}
for(int i=0;i<6;i++)
{
    if(largest<arr[i])
    {
        largest=arr[i];
    }
}
while(largest>0)
{
    nop++;
    largest=largest/10;
}

for(int pass=0;pass<nop;pass++)
{
    for(int i=0;i<10;i++)
    {
        bucketcount[i]=0;
    }
}
```



```

    for(int j=0;j<6;j++)
    {
        reminder=(arr[j]/divisor)%10;
        bucket[reminder][bucketcount[reminder]++]=arr[j];
    }
    int i=0;
    for(int k=0;k<10;k++)
    {
        for(int j=0;j<bucketcount[k];j++)
        {
            arr[i]=bucket[k][j];
            i++;
        }
    }
    divisor=divisor*10;
}
printf("after sorting\n");
for(int i=0;i<6;i++)
{
    printf("%d\n",arr[i]+min);
}

return 0;
}

```

OUTPUT:

```
before sorting
```

```
15
```

```
45
```

```
-1
```

```
2
```

```
231
```

```
480
```

```
after sorting
```

```
-1
```

```
2
```

```
15
```

```
45
```

```
231
```

```
480
```

10. Write a C program to copy one array to another using pointers.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr1[5] = {10, 20, 30, 40, 50};
```

```
    int arr2[5];
```

```
    int *p1 = arr1;
```

```
    int *p2 = arr2;
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        *(p2 + i) = *(p1 + i);
```

```
    }
```

```
    printf("Original array:\n");
```

```
for (int i = 0; i < 5; i++)
{
    printf("%d ", *(p1 + i));
}

printf("\nCopied array:\n");

for (int i = 0; i < 5; i++)
{
    printf("%d ", *(p2 + i));
}

return 0;
}
```

OUTPUT:

```
Original array:
10 20 30 40 50
Copied array:
10 20 30 40 50
```